

Micro-Rato 26'

Regras e especificações técnicas da modalidade

Explorer

1 Introdução

Este documento descreve as regras e especificações técnicas aplicáveis à *edição MicroRato 2026, chamada Explorer*.

A modalidade *Explorer* é uma competição robótica, que decorre num ambiente de simulação a correr numa rede de computadores. O sistema de simulação cria um labirinto virtual, que os robôs concorrentes têm de resolver. O labirinto é construído sobre uma grelha de células quadradas, sendo uma delas definida como a *célula inicial* e outra como a *célula alvo*.

O sistema de simulação também cria os corpos virtuais dos robôs. Todos os robôs virtuais têm o mesmo tipo de corpo. É composto por uma base circular, equipada com sensores, atuadores e botões de comando.

Os participantes devem fornecer o software, denominado *agente*, que controla o movimento de um robô virtual, para alcançar o objetivo da competição. O simulador estima as medidas dos sensores que são depois enviadas ao agente. Por outro lado, recebe e aplica ordens de atuação vindas do agente. Assim, o agente atua como o cérebro do robô.

O desafio na edição de 2026 do *MicroRato* é o seguinte. No início, o robô é colocado no centro de uma célula inicial desconhecida. Deve explorar o labirinto para localizar a célula-alvo, enquanto constrói uma representação do labirinto explorado até então. Depois, deve regressar à célula inicial, pelo caminho mais curto possível. O resultado depende do cumprimento dos objetivos de desafio e das penalizações sofridas.

2 Ambiente de simulação

O sistema virtual que suporta o *concurso MicroRato* baseia-se numa arquitetura distribuída, onde entram em jogo 3 tipos diferentes de aplicações: o simulador, o visualizador e os agentes (ver figura 1).

O simulador é responsável por:

- Implementar os corpos virtuais dos robôs.
- Estimar as medições dos sensores e enviá-las ao agente correspondente.
- Movendo robôs dentro do labirinto, de acordo com ordens recebidas do agente correspondente e tendo em conta as restrições do ambiente. Por exemplo, um robô não consegue atravessar uma parede.
- Atualização da pontuação do robô, tendo em conta os objetivos cumpridos e as penalizações aplicadas.
- Enviar partituras e posições dos robôs ao visualizador.
- Disponibilizando um painel de controlo para iniciar/reiniciar e parar a competição.

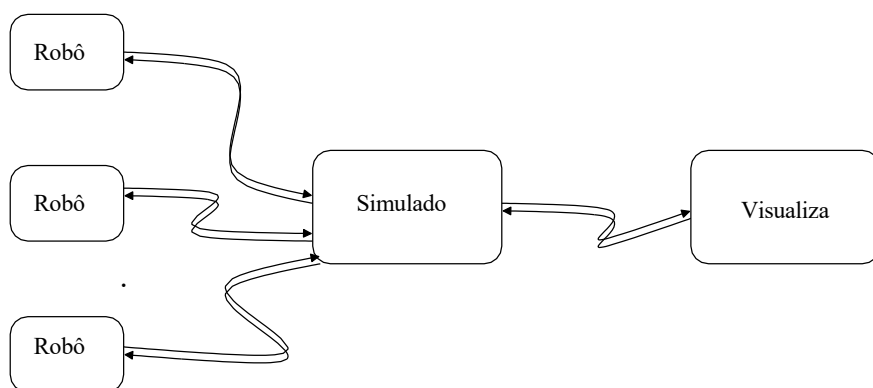


Figura 1: Visão geral do sistema de simulação.

O visualizador é responsável por:

- Mostrando graficamente robôs no labirinto de competição, incluindo os seus estados e pontuações.
- Disponibilizando um painel de controlo para iniciar/reiniciar e parar a competição.

O sistema de simulação é discreto e orientado pelo tempo. Em cada passo de tempo, o simulador envia medições dos sensores aos agentes, recebe ordens de atuação, aplica-as e atualiza as pontuações. Para a *edição MicroRato 2026*, o tempo de ciclo é de 50 milissegundos.

Todos os elementos em jogo, nomeadamente labirinto e robôs, são virtuais, pelo que não há necessidade de uma unidade de comprimento real. Assim, usamos u_m como unidade de comprimento, que corresponde ao diâmetro do robô. Todos os intervalos de tempo são medidos como múltiplos do tempo do ciclo. Denotamos u_t a nossa unidade de tempo, representando o tempo do ciclo.

3 Corpo do Robô

Os corpos dos robôs virtuais têm uma forma circular, com 1 u_m de largura, e estão equipados com sensores, atuadores e botões de comando (ver figura 2).

3.1 Sensores

Os elementos sensores em cada robô incluem: 4 sensores de obstáculos, 1 bússola, 1 para-choques (sensor de colisão), 1 sensor terrestre e um GPS. Alguns sensores estão sempre disponíveis, nomeadamente o para-choques e o GPS. Os outros — sensores terrestres, de obstáculos e bússola — estão disponíveis apenas mediante pedido, com um limite de 4 por ciclo.

Os modelos de sensores tentam representar dispositivos reais. Assim, por um lado, as suas medidas são ruidosas. Por outro lado, a leitura dos sensores é afetada por n unidades de tempo de latência, onde n depende do sensor em particular. Isto significa que os respetivos valores têm cerca de n ciclos de simulação, ou seja, quando um agente recebe um valor, representa uma medida realizada há n ciclos.

Segue-se uma descrição para cada tipo de sensor. Um resumo é fornecido na tabela 1.

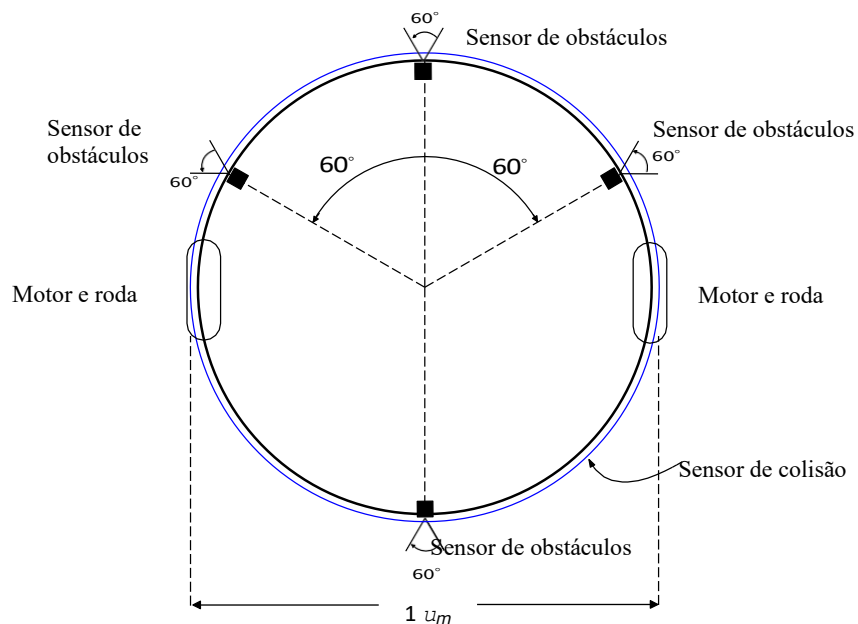


Figura 2: Corpo do robô virtual.

Tabela 1: Caracterização dos sensores. Os sensores a pedido são limitados a um máximo de 4 por ciclo.

Sensor	Distribuição	Resolução	Tipo de ruído	Desvio	Latência	A pedido
Sensor de obstáculos	[0.0, 100.0]	0.1	aditivo	0.1	0	sim
Bússola	[-180, +180]	1	aditivo	2.0	4	sim
GPS (posição)		0.1	aditivo	0.5	0	Não
Bumper	Sim/Não	 N/A			0	Não
Sensor de solo	Sim/Não	 N/A			0	sim

- **Sensores de obstáculos** medem distâncias entre o robô e os obstáculos circundantes, incluindo outros robôs. Têm posições pré-definidas, mas podem ser reposicionadas na inicialização do robô, embora apenas na zona do robô. A Figura 2 mostra as suas posições padrão.

Cada sensor tem um ângulo de abertura de 60 graus. A medida é inversamente proporcional à distância mais baixa aos obstáculos detetados, variando entre 0.0 e 100.0, com resolução de 0.1. O ruído é adicionado à medida ideal seguindo uma distribuição normal (gaussiana) com média 0 (zero) e desvio padrão 0.1. Os sensores de obstáculos têm uma latência de 0 unidades de tempo.

- A **bússola** está posicionada no centro do robô e mede a sua posição angular em relação ao *Norte virtual*. Assumimos que o eixo X (horizontal) está virado para o norte virtual.

As suas medidas variam de -180 a +180 graus, com resolução de 1 grau. O ruído é adicionado à medida ideal seguindo uma distribuição normal (gaussiana) com média 0 (zero) e desvio padrão 2.0. A bússola tem uma latência de 4 unidades de tempo.

- O **GPS** é um dispositivo que devolve a posição do robô no mundo, com resolução 0.1. Está localizado no centro de robôs. Quando a simulação começa, o labirinto é posicionado aleatoriamente no mundo, sendo as coordenadas de origem (canto esquerdo, inferior) atribuídas a um par de valores na gama 0–1000. O ruído pode ser adicionado às medidas ideais seguindo uma distribuição normal (gaussiana) com média 0 (zero) e desvio padrão 0.5. Tem uma latência de 0 unidades de tempo. O GPS **não está** disponível durante a competição, mas pode ser usado durante o desenvolvimento para testar o seu algoritmo de localização.
- O **para-choques** corresponde a um anel colocado à volta do robô. Funciona como uma variável booleana ativada sempre que há colisão, com uma latência de 0 unidades de tempo.
- O **sensor terrestre** é um dispositivo que deteta se o robô está completamente sobre a *área alvo*. Tem uma latência de 0 unidades de tempo.

3.2 Atuadores

O robô virtual tem 2 motores e 3 LEDs de sinalização (luzes). Os motores tentam representar, embora aproximadamente, motores reais. Assim, têm inércia e ruído. Segue-se uma descrição para cada tipo de atuador. Um resumo é fornecido na tabela 2.

- Os 2 **motores acionam** duas rodas, colocadas conforme mostrado na figura 2. O movimento do robô depende da potência aplicada aos dois motores. São possíveis tanto movimentos translacionais como rotacionais. Se forem aplicados os mesmos valores de potência

Tabela 2: Caracterização dos atuadores.

Atuador	Distribuição	Resolução	Tipo de ruído	Desvio padrão
Motor	[-0.15, +0.15]	0.001	multiplicativo	1.5%
<i>fim conduzido</i>	Liga/Desliga	 N/A		
<i>Liderado de regresso</i>	Liga/Desliga	 N/A		

para ambos os motores, o robô move-se ao longo do seu eixo frontal. Se os valores de potência forem simétricos, o robô roda em torno do seu centro.

A potência aceite pelos motores varia entre -0.15 e $+0.15$, com resolução 0.001 . No entanto, esta não é a potência aplicada às rodas devido à inércia e ao ruído. Consulte a secção 7 para uma descrição da relação potência entrada/saída, ou seja, a relação entre a potência solicitada pelos agentes e a potência aplicada às rodas. O ruído é multiplicativo, seguindo uma distribuição normal (gaussiana) com média e desvio padrão iguais a 1 e 1.5%, respetivamente.

Uma ordem de potência aplicada a um motor mantém-se em vigor até que seja dada uma nova ordem. Por exemplo, se um agente aplicar uma dada potência a um motor num determinado passo temporal, essa potência será aplicada continuamente nos passos seguintes até que o agente envie uma nova ordem de energia.

- Os 2 leds são chamados *return led* e *end led* e são usados para sinalizar a concretização dos objetivos. A forma como devem ser usados depende do desafio da competição. Consulte a secção 5 para mais detalhes.

3.3 Botões

Cada robô virtual está equipado com 2 botões, chamados *Start* e *Stop*. São usados pelo simulador para iniciar e interromper a competição. O botão *Start* é pressionado para iniciar uma competição ou para reiniciar uma que já foi interrompida. O botão *Stop* é pressionado quando uma competição é interrompida. Os agentes devem ler o estado destes botões e agir em conformidade.

4 Cenário de competição

O cenário de competição (ver figura 3 para um exemplo) é composto por uma grelha de células ao quadrado, delimitadas externamente, com uma *célula inicial* e uma *célula-alvo* no interior. Segmentos de parede podem ser colocados entre células adjacentes, para dificultar os movimentos dos robôs, criando o labirinto. A *célula-alvo* é a célula que deve ser alcançada pelo robô e tem um material de terra identificável pelo sensor de terra. A *célula inicial* é a célula de saída do robô e é indistinguível das outras, exceto da célula-alvo. São observadas as seguintes regras:

1. O lado de cada célula mede 2 um .
2. As dimensões máximas do labirinto têm 7 células de altura e 14 células de largura.

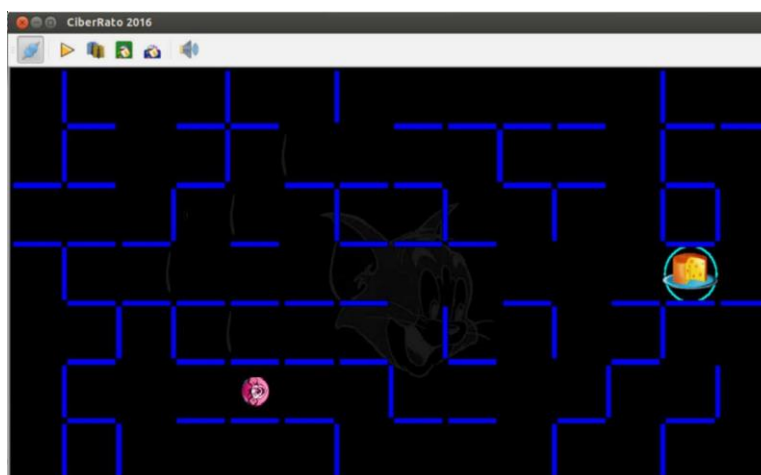


Figura 3: Um cenário de competição.

3. Todos os segmentos de parede são $0.1 \text{ } \mu\text{m}$ de largura.
4. A postura do robô no início é sempre 0 ou 180 graus.
5. Existe sempre um caminho possível do início até à célula-alvo.

5 Competição

5.1 Estrutura computacional

A competição decorre numa rede de dois computadores. Uma, normalmente a operar em Linux, é usada para executar o simulador e o visualizador. A outra é usada pelo participante para gerir o agente. Pode operar em Windows, Linux ou outro sistema operativo com pilha IP, ligação Ethernet e as bibliotecas apropriadas.

5.2 Desafio

O principal objetivo é o desenvolvimento de um agente robótico para comandar um robô móvel, fazendo-o mover-se para uma célula-alvo desconhecida num ambiente semi-estruturado e depois regressar à célula inicial pelo caminho mais curto possível.

Um robô pode visitar a célula-alvo várias vezes antes de decidir regressar à sua célula inicial. No entanto, quando decide fazê-lo, deve ligar o *LED de retorno* dentro da célula alvo. Depois, quando chegar à *célula inicial*, deve ligar o *LED final* e parar.

Para cumprir melhor os objetivos do desafio, os participantes devem estar cientes de que:

- A informação dos sensores de obstáculos tem de ser usada para evitar e seguir as paredes.
- A posição do alvo pode ser detetada usando o sensor terrestre, pois o valor medido por este sensor muda quando o robô está completamente dentro desta região.
- A célula de arranque não pode ser detetada por nenhum sensor especial.
- Não existem sensores codificadores nas rodas, mas a postura do robô pode ser estimada usando o modelo motor e os comandos de velocidade enviados para a simulação. A estrutura da grelha do labirinto pode também ser usada para melhorar a autolocalização.
- Para lidar com o ruído nos sensores, deve ser usado algum tipo de filtragem.
- Deve-se usar alguma forma de representar o ambiente ou os possíveis caminhos de navegação, para calcular o caminho mais curto até à célula inicial.

5.3 Estrutura da competição

A competição desenrola-se em 3 etapas. Na primeira e segunda mão, todas as equipas participam. As três equipas melhor qualificadas após a segunda mão vão para a final.

Em cada etapa, todas as equipas participam numa única prova. Em cada prova, o robô compete sozinho.

O cenário do jogo pode variar a cada rodada, mas é o mesmo em todas as provas ao longo de uma etapa. Os cenários são desconhecidos antecipadamente para as equipas.

5.4 Pontuação

No final de cada ensaio, é atribuída uma pontuação à equipa participante. A pontuação calculada tem em conta o cumprimento dos objetivos e as penalizações incorridas. São aplicadas as seguintes regras:

$$\text{Pontuação}[s] = T + \frac{S}{30} + P$$

T (Run Time): Tempo que o rato demora para sair da partida e chegar à linha de chegada na sua tentativa mais rápida. O cronómetro inicia quando o rato sai do quadrado (0,0) e para assim que os sensores detetam a célula-alvo.

S (Search Time): Tempo que o robô gasta para explorar o labirinto e mapear caminhos. Não são considerados os segundos que o robô fica parado no ponto de partida.

P (Penalties): Acréscimos de tempo por penalidades cometidas pelo rato. As penalizações são as seguintes:

- Toques na parede: 2 segundos por cada colisão.
- Reset Manual: Caso o utilizador pedir para reiniciar o percurso, são acrescentados mais 5 segundos.

O tempo total de cada equipa não deve exceder 10 minutos.

O número máximo de runs que o robô pode fazer é 5, ou seja, é o número máximo de vezes que pode voltar à base e ir à célula alvo em velocidade máxima.

Se o robô permanecer mais de 5 minutos sem realizar uma run, a prova é encerrada, pois considera-se que o algoritmo não é capaz de atingir o objetivo.

5.5 Classificação

É feita a media dos pontos da primeira e segunda rodada e as três equipas com menor tempo são qualificadas para a etapa final.

Apenas as últimas três equipas são classificadas de 1º a 3º lugar de acordo com a sua pontuação na etapa final.

5.6 Júri

O júri é a autoridade máxima em termos de interpretação e aplicação das regras. A sua missão é verificar o cumprimento das regras por robôs e ajudar o árbitro nas suas decisões.

Não pode recorrer das decisões do painel.

O painel é designado pela *MicroRato Organization*.

5.7 Árbitro

O árbitro controla a competição e assegura o cumprimento das regras do concurso. O árbitro pode interromper a competição para consultar o painel de juizes. Em todas as questões omitidas, deve, obrigatoriamente, consultar o painel de juizes.

O árbitro é designado pela *MicroRato Organization*.

5.8 Circunstâncias anormais

Como consequência de uma situação anormal, o árbitro pode interromper a competição a qualquer momento para consultar o painel de juizes. Quando isto acontece, todos os robôs são notificados através do *botão Stop* e são imobilizados na mira do simulador. O tempo também está congelado.

O painel pode decidir retomar, terminar ou repetir o julgamento atual. O processo de retomar uma competição previamente interrompida é controlado pelo árbitro, sendo os robôs notificados através do *botão Start*. As posições espaciais e angulares dos robôs na hora do reinício são exatamente as mesmas que tinham na hora da interrupção.

6 Parâmetros de simulação

A configuração do simulador para uma etapa faz-se passando-lhe os seguintes elementos:

- Tempo de ciclo e tempo total de competição.

simulação atual;

- `lNoisyOutPowt` e `rNoisyOutPowt` são ruído motor calculado aleatoriamente;
- `lNoisyOutPowt` e `rNoisyOutPowt` são os valores de potência a aplicar às rodas no instante t .

A abordagem de movimento implementada pelo simulador decompõe-o em dois componentes, um linear ao longo do eixo frontal do robô e outro rotacional em torno do seu centro. O simulador aplica primeiro o componente linear e depois o rotacional. Estas componentes são dadas pelas seguintes equações.

$$\begin{aligned} \text{lint} &= (\text{lNoisyOutPowt} + \text{rNoisyOutPowt}) / 2 \\ \text{roto} &= (\text{rNoisyOutPowt} - \text{lNoisyOutPowt}) / \text{diam} \end{aligned}$$

onde

- `lint`, dado em U_M , é o componente linear do movimento, no instante T ;
- `roto`, dada em radianos, é o componente rotacional do movimento, no instante t ;
- `Diam` é o diâmetro do robô;

Seguem-se os seguintes passos:

1. Uma nova posição do robô é calculada, com base em equações anteriores e assumindo que não existem obstáculos.
2. Se a nova posição implica uma colisão com um obstáculo (parede), o componente linear do movimento é ignorado, sendo apenas o componente rotativo aplicado,
3. Além disso, o sensor de colisão é ativado e a penalização de colisão é aplicada.

Podem aceder às ferramentas do simulador em <https://github.com/iris-ua/ciberRatoTools>, as instruções que estão no README.md são para o ubuntu 20.04, mas devem funcionar, eventualmente com poucas alterações, em versões mais recentes

8 Protocolos de Comunicação

A comunicação entre o simulador e os agentes baseia-se em *sockets* UDP, ou seja, as mensagens formatadas em estruturas XML. Existem 5 etiquetas de mensagem a considerar: *pedido de registo*, *resposta a concessão*, *resposta de recusa*, *dados do sensor* e *ordem de atuação*. Só precisa de ler esta secção se planeia usar uma linguagem de programação diferente de C, C++ ou Java. Caso contrário, pode usar as bibliotecas de funções disponíveis no pacote de ferramentas (`RobSock.h` para C/C++, `ciberIF.java` para Java e `croblink.py` para python).

8.1 Registo

Cada agente deve registar-se no simulador enviando uma *mensagem de pedido de registo* para a porta 6000 do endereço IP do computador que executa o simulador. A mensagem parece ser

```
<Robot Name="name">
  <IRSensor Id="sid" Angle="sangle"/>
```

```
</Robot>
```

onde o `nome` é o nome do robô, aquele que aparece no placar, `sid` é o ID de um sensor de obstáculo, variando de 0 a 3, e `sangle` é a posição angular do sensor na periferia do robô, variando de $-180,0$ a $+180,0$. As etiquetas `IRSensor` são opcionais. Tens de os usar se quiseses mudar a posição padrão dos sensores de obstáculos.

Se o simulador recusar o pedido de registo, envia a mensagem ao agente

```
<Reply Status="Refused"></Reply>
```

Se o simulador aceitar o pedido de registo, envia ao agente a mensagem

```
<Reply Status="Ok">
```

```
<Parameters SimTime="time" CycleTime="time"
  CompassNoise="noise" ObstacleNoise="noise" MotorsNoise="noise" />
</Reply>
```

onde o `time` é um valor inteiro, representando um tempo, em ut , e o `noise` é um valor real, representando um nível de ruído. O agente deve memorizar a porta de onde veio essa resposta e enviar todas as novas mensagens para lá.

8.2 Ordens de ativação

Em cada ciclo, os agentes podem enviar ao simulador 1 ou mais ordens de atuação. No entanto, o número de encomendas por dispositivo está limitado a uma. Se mais do que uma for recebida, apenas a última recebida será considerada. Cada *mensagem de ordem de atuação* é um subconjunto de onde `pow` é um valor real, representando um poder, e `act` é a palavra "On" ou "Off", representando uma ordem para ligar ou desligar um *LED*. O número de pedidos de sensores por ciclo é limitado a 4 — se forem solicitados mais, apenas 4, escolhidos arbitrariamente, são considerados. As ordens motoras são persistentes, no sentido em que a ordem é mantida até que uma nova seja recebida pelo simulador. Os pedidos dos sensores não são persistentes. Se um agente quiser ler os mesmos sensores em dois ou mais ciclos consecutivos, precisa de enviar os pedidos dos sensores em cada ciclo.

```
<Actions LeftMotor="pow" RightMotor="pow"
  VisitingLed="act" ReturningLed="act" EndLed="act"
  <SensorRequests IRSensor0="Yes" IRSensor1="Yes" ...
    Ground="Yes" Compass="Yes"/>
</Actions>
```

8.3 Dados dos sensores

Após o registo, o simulador, em cada ciclo, envia ao robô uma mensagem com dados do sensor. A mensagem enviada é um subconjunto da que é mostrada abaixo, sendo o subconjunto dependente dos pedidos dos sensores recebidos.

```
<Measures Time="time">
  <Sensors Compass="angle" Collision="yesno" Ground="groundid">
    <IRSensor Id="0" Value="irmeasure"/>
    <IRSensor Id="1" Value="irmeasure"/>
    <IRSensor Id="2" Value="irmeasure"/>
    <IRSensor Id="3" Value="irmeasure"/>
    <GPS X="coord" Y="coord"/>
  </Sensors>
  <Leds EndLed="onoff" VisitingLed="onoff" ReturningLed="onoff"/>
  <Buttons Start="onoff" Stop="onoff"/>
</Measures>
```

onde `time` é um valor inteiro que representa o tempo atual, `angle` é um valor real que representa um ângulo, em radianos, `yesno` é a palavra "Sim" ou "Não", `groundid` é 0 se o robô estiver completamente dentro da *célula alvo* e -1 caso contrário, `irmeasure` é um número real que representa uma medida de sensor de obstáculo, `coord` é um número real que representa uma coordenada espacial GPS e `onoff` é a palavra "On" ou "Off" que representa o estado de um *LED* ou botão.